

SECURITY & LOGIC AUDIT · INTERNAL REVIEW

تقرير مراجعة تطبيق الزيارات الميدانية

مراجعة شاملة لمنطق العمل وصلاحيات المستخدمين على مستوى التطبيق وقاعدة بيانات Odoo — رصد المشاكل الحالية مع حل مقترح لكل مشكلة.

MODEL

x_dh_visit

BACKEND

Odoo 19 · SaaS

PROJECT

DH Visits

DATE

2026-06-25

أُعِدَّ هذا التقرير بعد فحص مباشر لقاعدة البيانات الحيّة (ACL · Record Rules · Views · Server Actions) إضافةً إلى مراجعة الكود (Flutter). جاهز للعرض على Team Lead.

الملخص التنفيذي

التطبيق سليم وظيفياً من ناحية الواجهة، لكن المراجعة كشفت عن **خلل جوهري في نموذج الصلاحيات (Authorization Model)** على مستوى قاعدة بيانات Odoo: أي **موظف** يدخل على واجهة Odoo يمتلك صلاحيات تفوق دوره — فيقدر **ينشئ** زيارة لنفسه، و**يعدّل** كل تفاصيلها، و**يعتمد** نفسه (Draft → Approved). هذا يكسر منطق العمل المتفق عليه: **المدير يجدول ويعتمد، والموظف ينقذ ويوثّق فقط**. كما رُصدت فجوات في سلامة البيانات (حقوق غير إجبارية، غياب سجل التدقيق Audit Trail) وفرص تحسين احترافية.

3

تحسينات احترافية

5

متوسطة (سلامة البيانات)

1

معماري (تضارب أدوار)

3

درجة / عالية الأولوية

جدول المشاكل المرصودة

#	المشكلة	الفئة	الخطورة
1	الموظف يقدر ينشئ زيارة لنفسه من Odoo	صلاحيات	درجة
2	الموظف يقدر يعدّل كل تفاصيل الزيارة (عميل/وقت/مندوب)	صلاحيات	درجة
3	الموظف يقدر يعتمد/يرفض زيارته بنفسه	صلاحيات	درجة
4	تضارب تعريف «المدير» بين السيرفر والتطبيق	معماري	عالية
5	غياب الحقوق الإجبارية (عميل/مندوب/تاريخ)	سلامة بيانات	متوسطة
6	الرفض بدون سبب إجباري	سلامة بيانات	متوسطة
7	غياب سجل التدقيق (من اعتمد ومتى)	تدقيق	متوسطة
8	لا يوجد منع لأكثر من زيارة مفتوحة (server-side)	سلامة بيانات	متوسطة
9	الزيارة المعتمدة تظل قابلة للتعديل	سلامة بيانات	متوسطة
10	ازدواج حقلي المندوب (x_user_id / x_employee_id)	تحسين	منخفضة
11	لا يوجد رقم مرجعي للزيارة (Sequence)	تحسين	منخفضة

#	المشكلة	الفئة	الخطورة
12	ضعف شاشة البحث/الفلاتر وغياب الإشعارات	تحسين	منخفضة

منهجية الفحص: تمّت المصادقة على قاعدة البيانات الحيّة كـ `admin` وقراءة الإعدادات الفعلية عبر `JSON-RPC` من الجداول التقنية: `ir.model.access · ir.rule · ir.ui.view · ir.actions.server · ir.model.fields`، بالتوازي مع مراجعة كود طبقة البيانات في التطبيق (`visits_repository.dart`, `auth_repository.dart`).

المشاكل الحرجة في الصلاحيات

درجة

الموظف يقدر ينشئ زيارة لنفسه من واجهة Odoo

1

• الوصف

عند الدخول على Odoo بحساب موظف، يظهر زر **New** ويستطيع إنشاء زيارة وإسنادها لنفسه. منطقياً، إنشاء/جدولة الزيارات مسؤولية المدير فقط، والموظف ينقذ ما هو مُسند إليه.

• السبب الجذري (تقني)

صلاحية الـ ACL لمجموعة المستخدمين مفعّل فيها Create، وكذلك قاعدة السجل الخاصة بالموظف:

```
// ir.model.access - x_dh_visit
x_dh_visit.user    group=Role/User    read=1 write=1 create=1 unlink=0

// ir.rule - "Visit: own (salesperson)"
domain = [('x_user_id','=',user.id)]    R=1 W=1 C=1 D=1
```

• الأثر

زيارات «وهمية» يصنعها الموظف لنفسه، تتجاوز جدولة المدير وتفسد دقة تقارير الأداء والتغطية.

• الحل المقترح (تأمين كامل)

- تعطيل create في الـ ACL لمجموعة Role/User = 0
- تعطيل create في قاعدة السجل «own» = 0
- الإبقاء على صلاحية الإنشاء لمجموعة **Visit Manager** فقط.
- لا يتأثر فلو التطبيق: الموظف يعمل Check-in عبر **تعديل (write)** زيارة جاهزة لا عبر إنشاء.

● الوصف

داخل تفاصيل الزيارة يستطيع الموظف تغيير **العميل**، **التاريخ**، **المنسوب**، **الفسند**، **النوع**، **بل والحالة**. المفروض أن يقتصر دوره على كتابة **الملاحظات / تقرير الزيارة** فقط، وبيانات الـ Check-in/out الآتية تلقائياً من التطبيق.

● السبب الجذري (تقني)

جميع الحقول قابلة للكتابة (readonly=0)، وقاعدة «own» تمنح write، والفورم لا يحتوي أي قفل مبني على الدور:

```
// ir.model.fields - كل الحقول الوظيفية
x_partner_id      readonly=0  x_user_id      readonly=0
x_visit_date      readonly=0  x_visit_type_id  readonly=0
x_state           readonly=0  x_check_in_time  readonly=0 ...
```

● الأثر

الموظف يقدر يعيد إسناد الزيارة لنفسه، يغيّر العميل أو الوقت بعد التنفيذ، فيفقد السجل مصداقيته كدليل ميداني.

● الحل المقترح (تأمين كامل: سيرفر + واجهة)

- إضافة حقل محسوب x_is_manager (يعتمد على (user.has_group('Visit Manager')).
- في الفورم: الحقول الحساسة "readonly=not x_is_manager" (العميل/المنسوب/التاريخ/النوع/الحالة)، وتبقى **الملاحظات** مفتوحة للموظف.
- تعزيز على السيرفر: Automated Action (on write) تمنع أي تعديل من غير المدير على الحقول المحمية وتُطلق خطأ — حتى لا يتجاوزها RPC المباشر.

• الوصف

يستطيع الموظف نقل الحالة من Draft → Approved بنفسه عبر أزرار الهيدر أو شريط الحالة. الاعتماد/الرفض قرار **المدير** بعد المراجعة، لا الموظف.

• السبب الجذري (تقني)

أزرار **Approve/Reject** في الهيدر مُعرّفة بدون groups (مرئية للجميع)، وال server actions التي تنقّذها group_ids فيها فارغة، ولا توجد قاعدة تمنع كتابة 'x_state='approved':

```
// ir.ui.view (form header) - بدون أي قيد مجموعات
<button name="413" string="Approve" invisible="x_state != 'submitted'"/>
<button name="414" string="Reject" invisible="x_state != 'submitted'"/>

// ir.actions.server
Visit: Approve    group_ids = []    code: r['x_state']='approved'
Visit: Reject     group_ids = []    code: r['x_state']='rejected'
```

• الأثر

انهيار حوكمة الاعتماد بالكامل: الموظف يعتمد عمله بنفسه، وتفقد دورة الموافقة قيمتها.

• الحل المقترح (تأمين كامل)

- إضافة groups="Visit Manager" لزرّي Approve/Reject في الفورم.
- ضبط group_ids ل server actions على **Visit Manager** فقط.
- جعل شريط الحالة x_state غير قابل للنقر (readonly) لغير المدير.
- قاعدة/Automated Action على السيرفر تمنع غير المدير من تحويل الحالة إلى approved/rejected (حماية ضد RPC).

● الوصف

السيرفر يعرّف المدير بعضوية مجموعة **Visit Manager**. بينما التطبيق يحدد المدير من علامتي `is_admin/is_system` فقط ولا يقرأ عضوية المجموعة. النتيجة: لو أضفنا مدير حقيقي للمجموعة، السيرفر يريه كل الزيارات، لكن **التطبيق يفتح له واجهة الموظف**. حالياً الأمران يتطابقان بالصدفة لأن `admin` فقط هو العضو.

● السبب الجذري (تقني)

```
// auth user (Flutter)
isManager = json['is_manager'] || isAdmin || isSystem // لا يقرأ مجموعة Visit Manager
// السيرفر: record rule "Visit: all (manager)" => group Visit Manager (id 37)
```

● الأثر

استحالة تعيين «مدير زيارات» غير الـ `admin` بشكل صحيح — عائق أمام التوسع وتعدّد المديرين.

● الحل المقترح

- عند تسجيل الدخول: استدعاء (`res.users.has_group('Visit Manager')`) وضبط `is_manager` بناءً عليه.
- توحيد «مصدر الحقيقة» للدور = عضوية مجموعة **Visit Manager** في الجانبين.

سلامة البيانات والتدقيق

متوسطة

غياب الحقول الإجبارية

5

• الوصف & السبب

حقول جوهرية مثل العميل والمندوب والتاريخ كلها `required=0`، فيمكن حفظ زيارة ناقصة بلا عميل أو بلا مندوب.

```
x_partner_id required=0    x_user_id required=0    x_visit_date required=0
```

• الحل المقترح

- ضبط `required=1` على `x_partner_id` `x_user_id` `x_visit_date` (نموذجياً + في الفورم).

متوسطة

الرفض بدون سبب إجباري

6

• الوصف

حقل `x_manager_note` موجود لكنه غير مطلوب عند الرفض، فيرفض العمل دون أن يعرف الموظف السبب.

• الحل المقترح

- اشتراط ملء `x_manager_note` قبل تنفيذ `Reject` (تحقق في الـ `server action` / نافذة سبب).
- عرض سبب الرفض بوضوح في شاشة الموظف داخل التطبيق.

متوسطة

غياب سجل التدقيق (Audit Trail)

7

• الوصف

لا يوجد تسجيل لمن اعتمد/رفض الزيارة ومتى (غير `write_uid` العام غير الدقيق دلاليًا).

• الحل المقترح

- إضافة حقول: `x_approved_by` (مستخدم) `x_decision_date` (تاريخ ووقت) ثُملاً تلقائياً عند الاعتماد/الرفض.
- تفعيل `mail.thread` (Chatter) على الموديل لتتبع كامل التغييرات.

● الوصف

التطبيق يمنع وجود زيارتين مفتوحتين للمندوب على مستوى الواجهة فقط؛ السيرفر لا يفرض ذلك، وفي حالة تزامن (race) قد ينتهي الأمر بسجلين Check-in مفتوحين.

● الحل المقترح

- قيد منطقي (Constraint / Automated Action) يمنع أكثر من زيارة بحالة Check-in مفتوحة لنفس `x_user_id`.

● الوصف

بعد الاعتماد لا يوجد قفل؛ فيمكن تعديل بيانات زيارة مُعتمدة وكأنها سجل حي، بينما يفترض أن تصبح سجلاً أرشيفياً للقراءة.

● الحل المقترح

- قفل كل الحقول (لجميع عدا أدوار محددة) عندما `x_state in ('approved', 'rejected')`.

تحسينات احترافية مقترحة

10

ازدواج حقلي المندوب

منخفضة

يوجد حقلان: `x_user_id` (المستخدم فعلياً) `x_employee_id` (فارغ وغير مستخدم في التطبيق) — يستبان التباساً في الفورم.

• الحل

- إخفاء/إزالة `x_employee_id` أو اشتقاؤه تلقائياً من `x_user_id` لربط الحضور (Attendance) فقط.

11

لا يوجد رقم مرجعي للزيارة (Sequence)

منخفضة

قيمة `x_name =` اسم العميل، والتطبيق يشتق `VIS/00012` محلياً فقط دون مرجع حقيقي مخزن.

• الحل

- إضافة `ir.sequence` لتوليد رقم زيارة فريد ثابت (`VIS/2026/00001`) يظهر في القوائم والتقارير.

12

شاشة البحث/الفلاتر والإشعارات

منخفضة

شاشة البحث في Odoo بسيطة بلا فلاتر جاهزة بالحالة أو تجميع، ولا توجد إشعارات على تغيّر الحالة.

• الحل

- فلاتر جاهزة (Draft / Submitted / Approved / Rejected) Group-by بالمندوب/العميل/التاريخ.
- إشعار للمدير عند `submitted`، وإشعار للموظف عند `approved/rejected` (Push أو Activity).
- منطقة Geofence إجبارية للـ Check-in (موجودة كمؤشر بصري لكنها غير مُلزمة).

خارطة طريق التنفيذ المقترحة

المرحلة	النطاق	المشاكل المعالجة	آلية التنفيذ
Phase 1	إغلاق الثغرات الحرجة في الصلاحيات (تأمين كامل سيرفر + واجهة)	4 · 3 · 2 · 1	+ Odoo على RPC تعديل Flutter بسيط

المرحلة	النطاق	المشاكل المعالجة	آلية التنفيذ
Phase 2	سلامة البيانات والتدقيق	9 · 8 · 7 · 6 · 5	Fields + Constraints + Automated Actions
Phase 3	التحسينات الاحترافية وتجربة الاستخدام	12 · 11 · 10	Sequence + Views + Notifications

ملاحظة تنفيذية: كل التعديلات تُنفَّذ على Odoo SaaS عبر JSON-RPC على الجداول التقنية (أسلوب Studio) دون رفع موديول، وكلها **قابلة للتراجع**. لا يتأثر فلو التطبيق الحالي للموظف لأنه يعتمد على **تعديل (write)** زيارة جاهزة لا على الإنشاء.